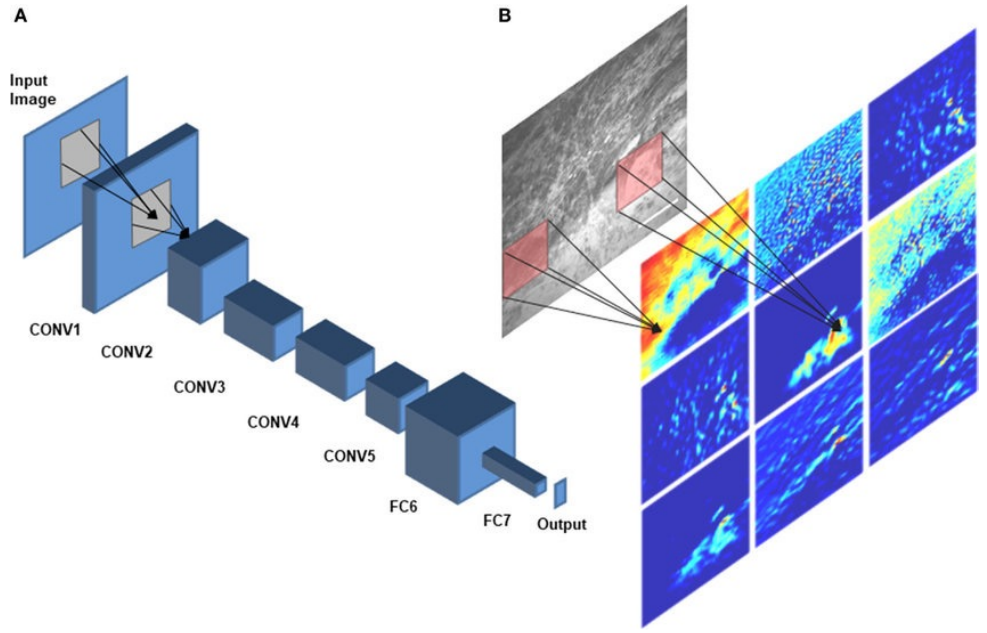
HSLU Lucerne University of Applied Sciences and Arts

Convolutional Neural Networks for Image Classification



20. März 2025

FH Zentralschweiz

ImageNet Challenge

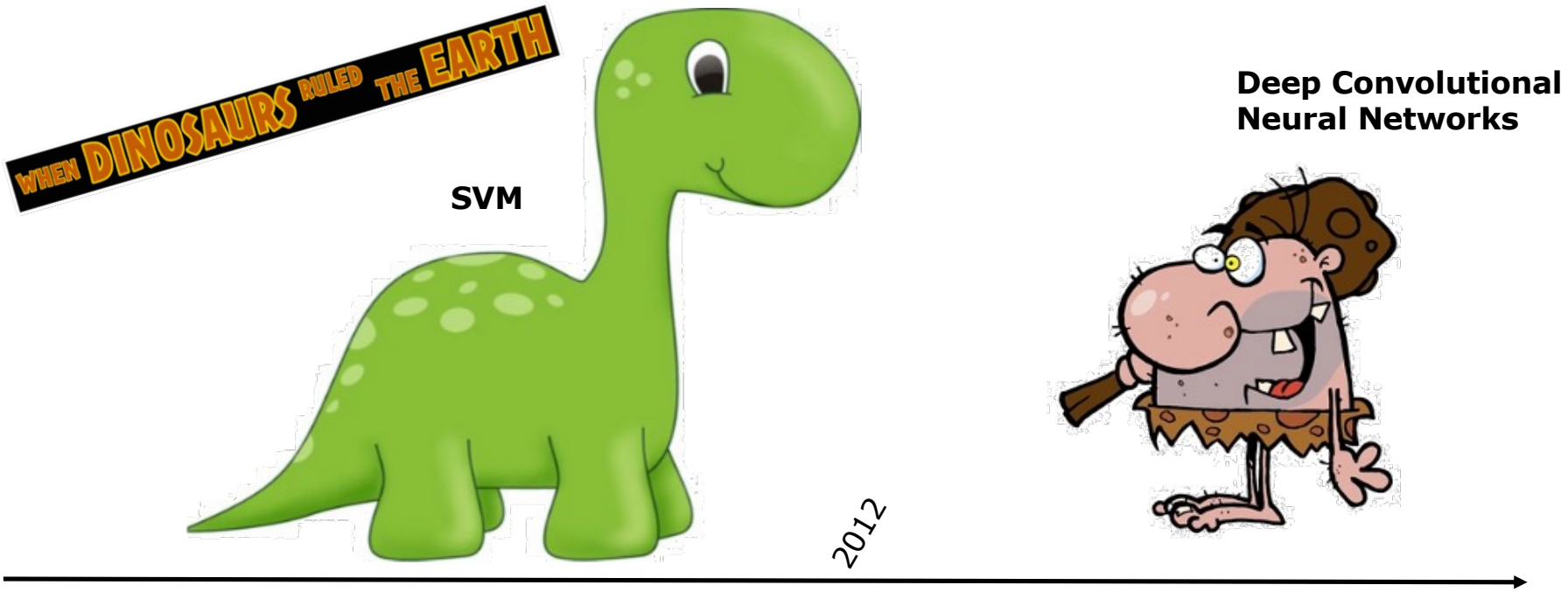


Image Net Challenge

Data Source:

- 14'197'122 Images organized in WordNet Hierarchy (nouns)
- 21'841 synsets (synonyms)

- For example 3822 synsets in animals with 732 images per set in average

- Challenge on 1000 classes
- One label per image (original challenge)
- Scoring on Top-1 and Top-5 errors
- (Done as challenge until 2017, now still used as benchmark)

Herb, herbaceous plant

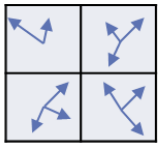
A plant lacking a permanent woody stem; many are flowering garden plants or potherbs; some having medicinal properties; some are pests

1400 pictures | 75.29% Popularity Percentile | Wordnet IDs

Olga Russakovsky*, Jia Deng*, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg and Li Fei-Fei. (* = equal contribution) **ImageNet Large Scale Visual Recognition Challenge**. *IJCV*, 2015.

Classic ML Approaches: Hand-designed Features + SVM

- Features calculate properties of an image or an image region
- Examples
  - Hand-designed filters (difference of Gaussians)
  - Histograms of pixel values
  - SIFT or HOG
- Goal:
  - Feature vector as a robust (e.g., to deformations or illumination) low-dimensional description of image

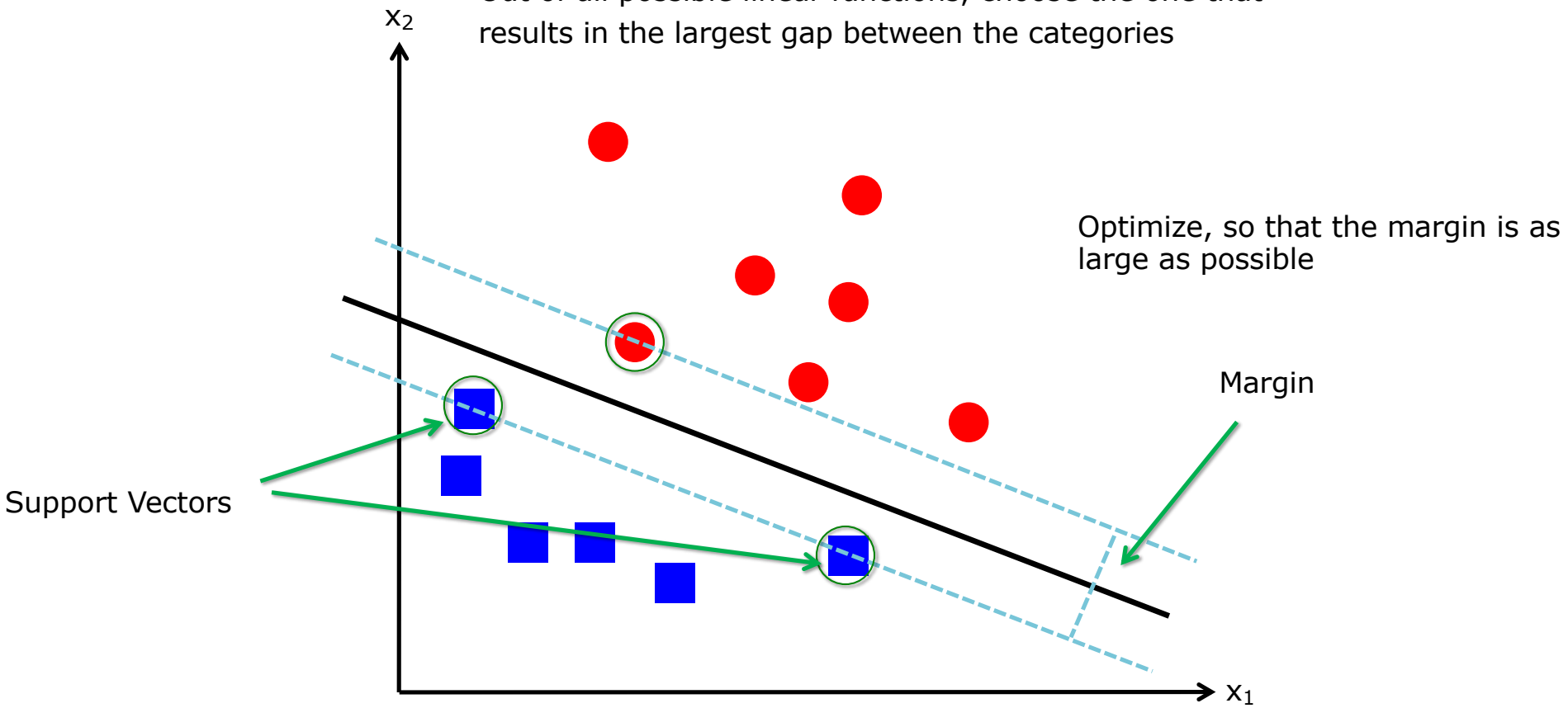


(0.2, 0.7, 0.3, ...)

Feature Vector

Support Vector Machine (SVM)

Out of all possible linear functions, choose the one that results in the largest gap between the categories



CNNs for image classification

Neural Network layer types:

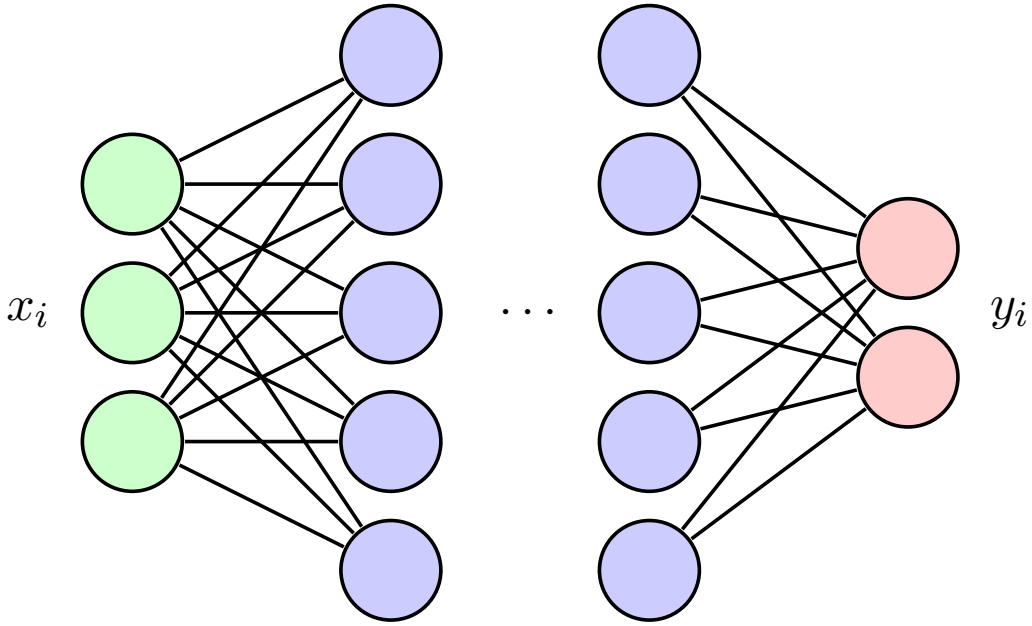
- MLP / Dense Layers
- Convolutional layers
- Pooling layers

- CNN Architectures
- Training large neural networks

Network types: Multi Layer Perceptron

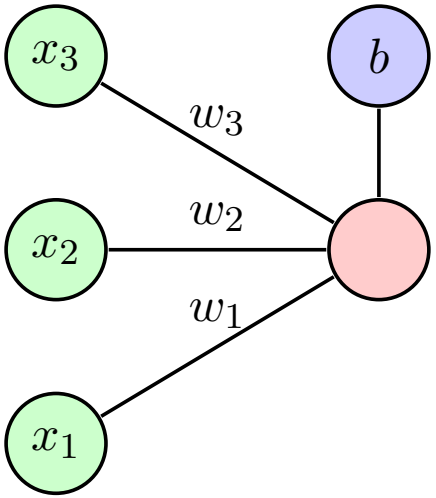
MLP or Dense Network

- n-dim Input x
- m-dim Output y
- Number of hidden layers
- All nodes between neighboring layers are connected



MLP

- Each node calculates its output based in the inputs, the weights (on the edges), a bias value and the activation function



$$y = \sigma \sum_{k=1}^{n} w_k x_k + b$$

pytorch

Linear

CLASS torch.nn.Linear(*in_features*, *out_features*, *bias=True*, *device=None*, *dtype=None*) [SOURCE]

Applies an affine linear transformation to the incoming data: $y = xA^T + b$.

pytorch: Activation Functions

ReLU

CLASS torch.nn.ReLU(*inplace=False*) [SOURCE]

Applies the rectified linear unit function element-wise.

$$\text{ReLU}(x) = (x)^+ = \max(0, x)$$

LeakyReLU

CLASS torch.nn.LeakyReLU(*negative_slope=0.01*, *inplace=False*) [SOURCE]

Applies the LeakyReLU function element-wise.

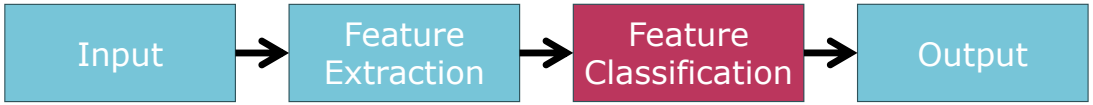
$$\text{LeakyReLU}(x) = \max(0, x) + \text{negative\_slope} * \min(0, x)$$

or

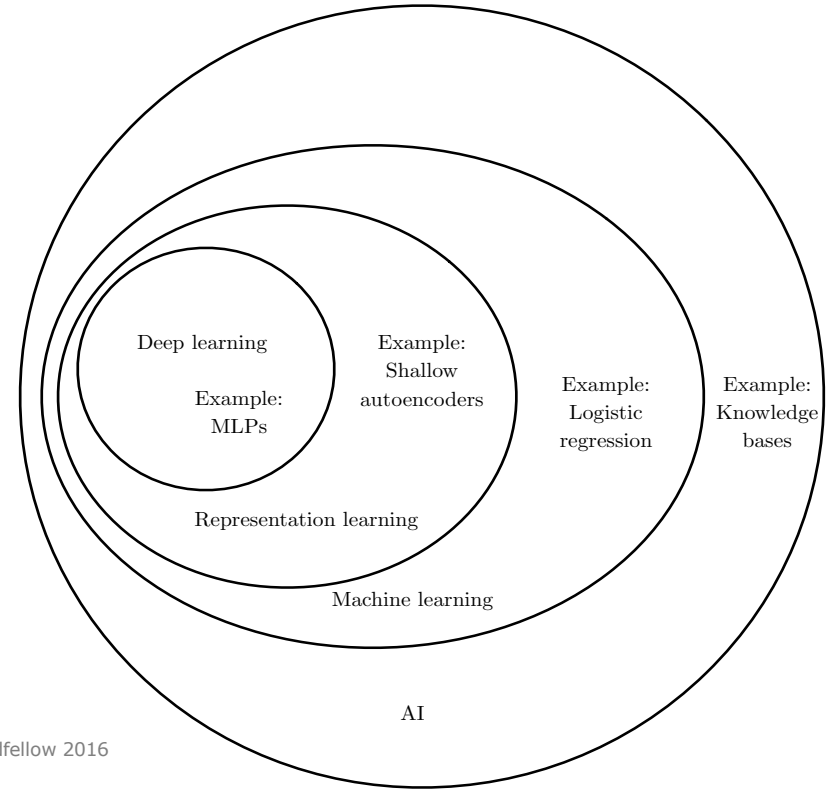
$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \text{negative\_slope} \times x, & \text{otherwise} \end{cases}$$

MLP: Applications

- Classification/Regression: Output layers after feature extractions
- Feature transformation (for example after attention layers)
- Dimensionality reduction
- ...

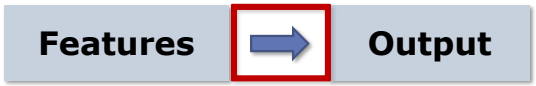


Deeper Neural Networks



Goodfellow 2016

Neural networks:



Deeper Neural networks:

Deep Learning = Representation Learning

Network Types: Convolutional Neural Networks (CNNs)

Building blocks are convolutions:

$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

$$(f * g)[n] = \sum_{m=-M}^{M} f[n - m] g[m]$$

in CNNs, one of the functions is given by a weight matrix:

$$g(x, y) = \omega * f(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} \omega(i, j) f(x - i, y - j),$$

Convolutional Neural Networks

- Usually cross correlation is used in CNNs APIs

$$g(x,y) = \omega \star f(x,y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} \omega(i,j) f(x+i, y+j),$$

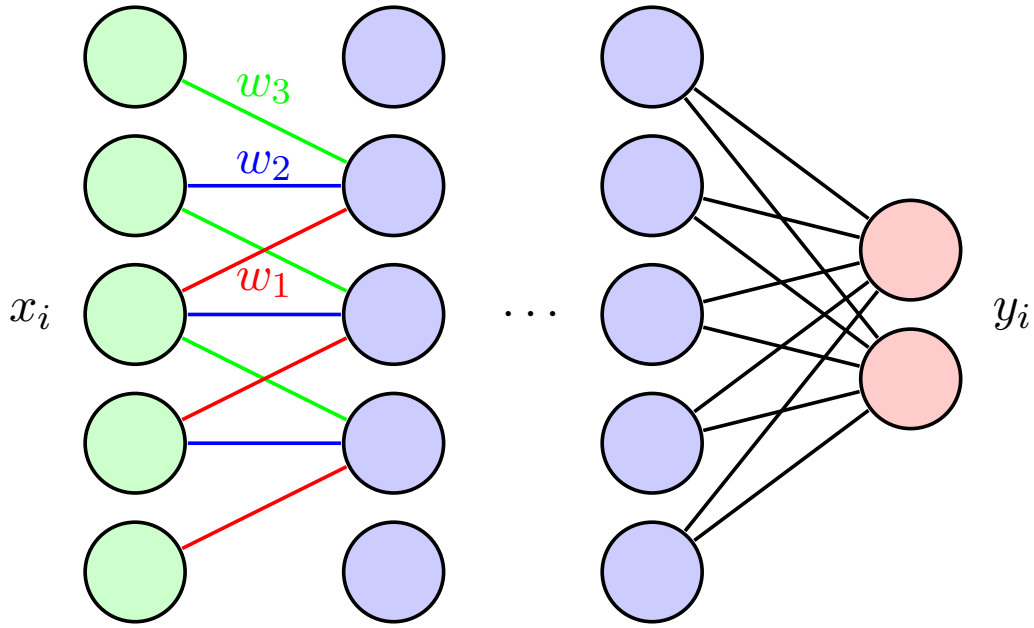
  - i.e. + instead of – or the kernel is not mirrored...
  - If the weights are learned through training, it does not matter

Convolutional Neural Networks

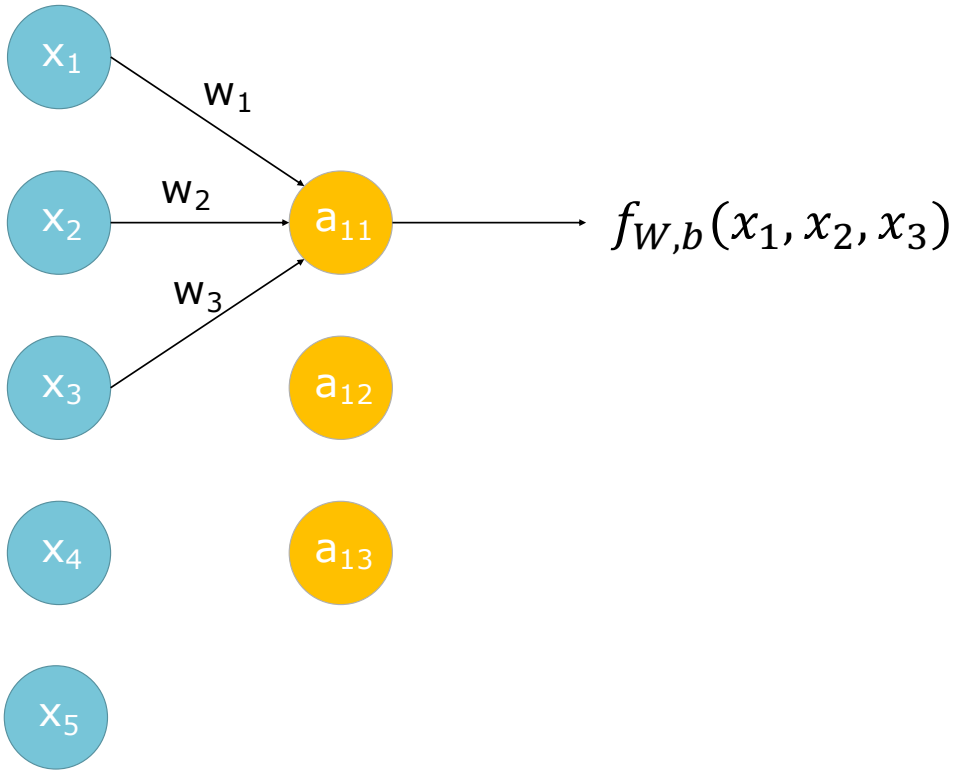
Original idea from image processing, where convolution (filters) are applied for example for feature selection.

Properties:

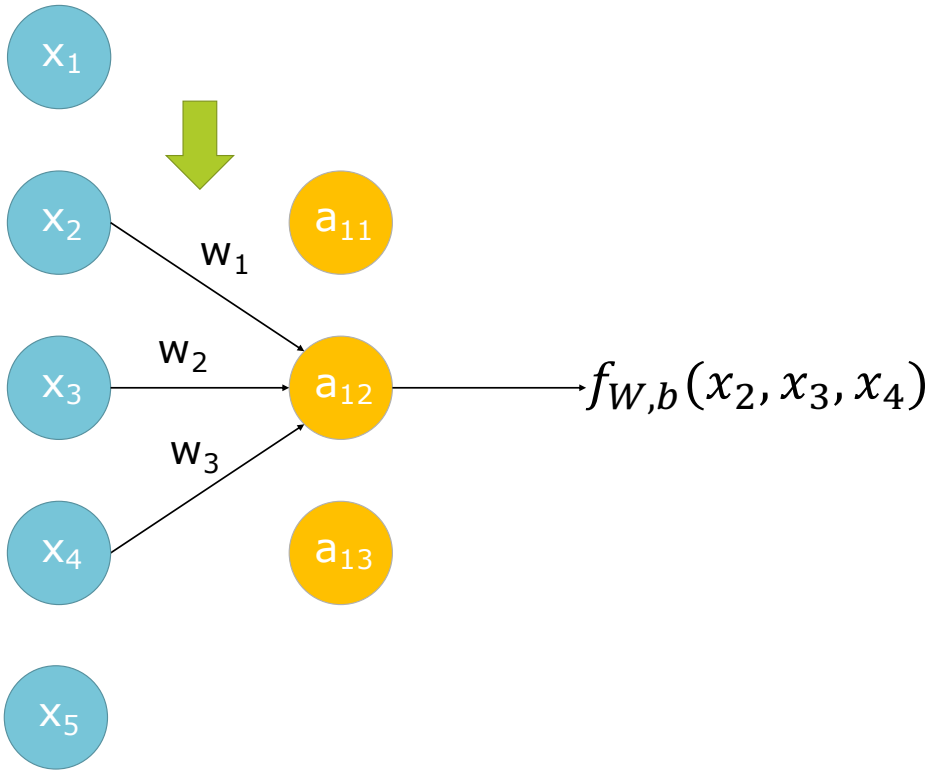
- Weights (parameters) are shared
- Connections are sparse
- Each node in a layer has a receptive field



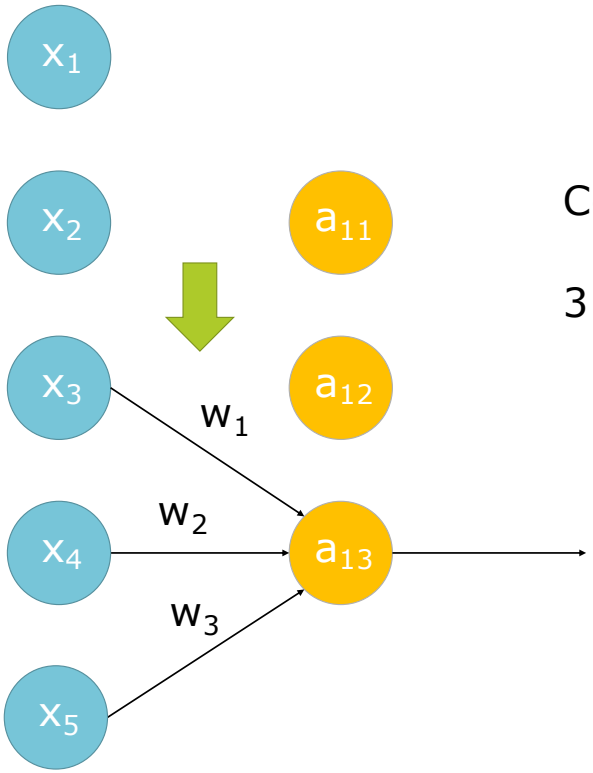
Convolutional Neural Networks



Convolutional Neural Networks



Convolutional Neural Networks

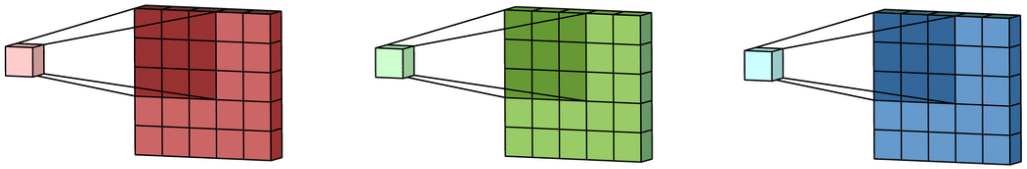


Convolutional neural network:

3 connections / parameters

$$f_{W,b}(x_3, x_4, x_5)$$

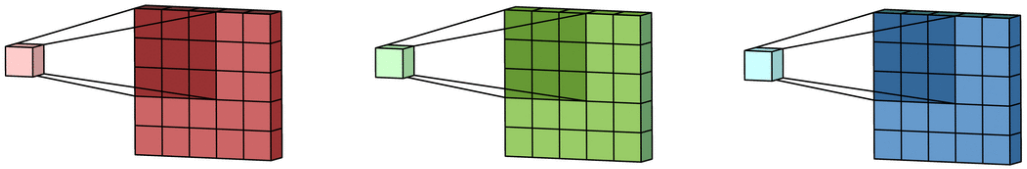
CNNs: Multiple channels

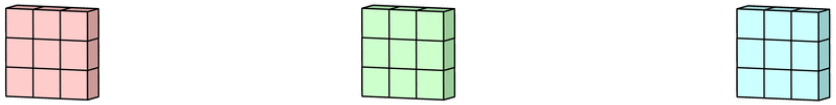


- When the input to the layer has multiple channels, a (different) 2D Kernel is applied to each channel

https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1

CNNs: Multiple channels





- The resulting channels are added
- (in an API, this is only one operation)

pytorch

Conv2d

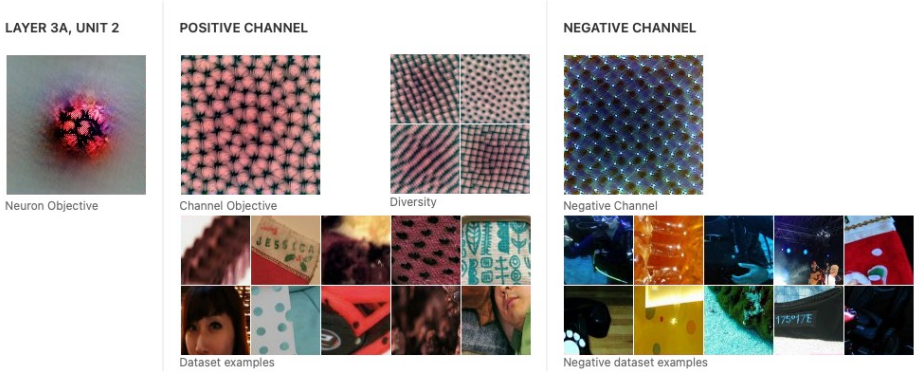
CLASS torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, *groups=1*, *bias=True*, *padding_mode='zeros'*, *device=None*, *dtype=None*) [SOURCE]

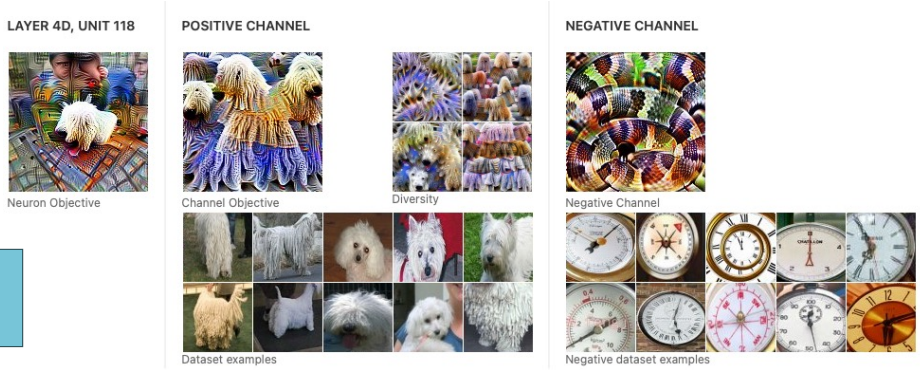
$$\text{out}\left(N_i, C_{\text{out}_j}\right) = \text{bias}\left(C_{\text{out}_j}\right) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}\left(C_{\text{out}_j}, k\right) \star \text{input}\left(N_i, k\right)$$

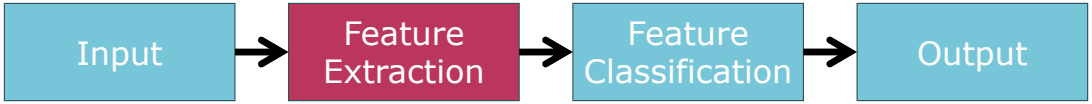
Why CNN layers?

Deep Learning is **representation** learning:

- CNN layers extract features or patterns in the input signal



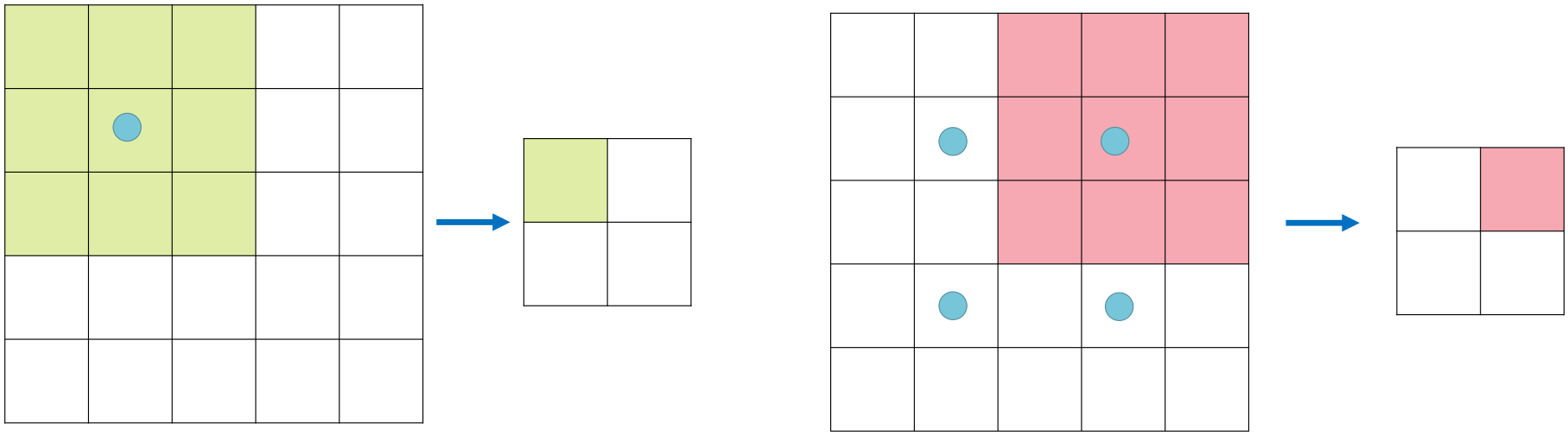




https://distill.pub/2017/feature-visualization/

Decrease Image Size: Strided Convolutions

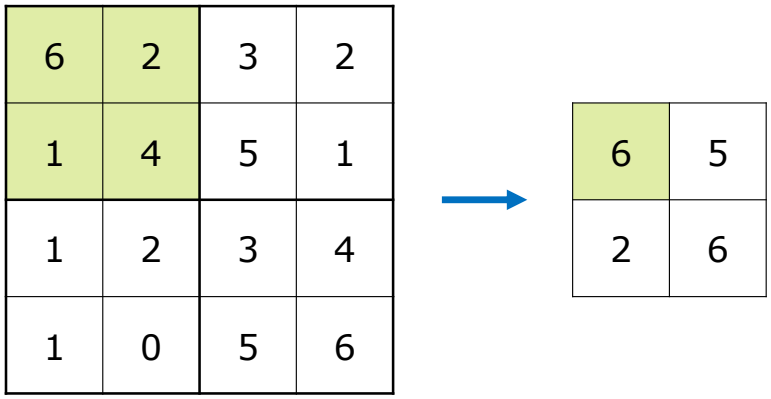
Strided 3x3 convolution with stride = 2



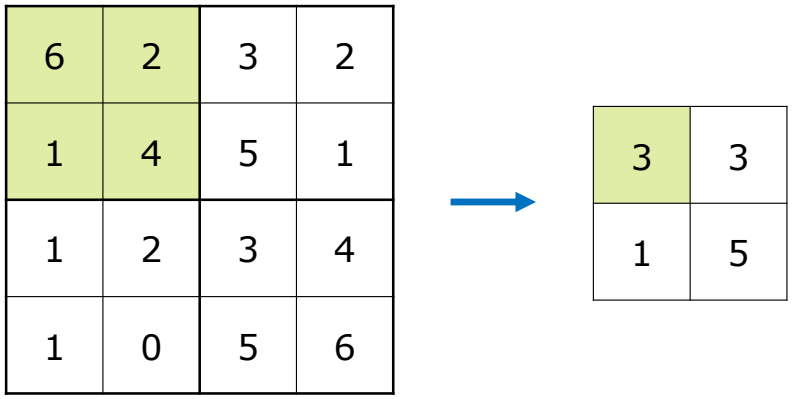
Filter moves 2 positions in **input** for every position in **output**

Decrease Image Size: Pooling

Max Pooling



Average Pooling



pytorch

MaxPool2d

CLASS torch.nn.MaxPool2d(*kernel_size*, *stride=None*, *padding=0*, *dilation=1*, *return_indices=False*, *ceil_mode=False*) [SOURCE]

Applies a 2D max pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W), output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

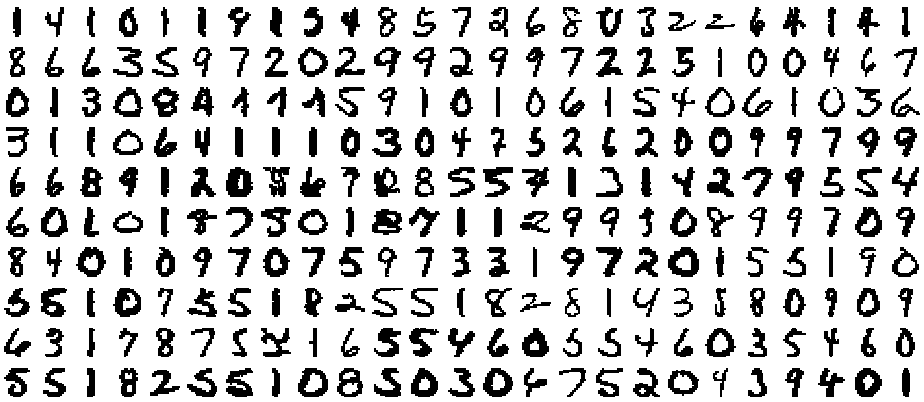
$$out(N_i, C_j, h, w) = \max_{m=0,\ldots,kH-1} \max_{n=0,\ldots,kW-1} \text{input}(N_i, C_j, \text{stride}[0] \times h + m, \text{stride}[1] \times w + n)$$

Deep Learning Architectures

- Training deep CNNs is difficult and over time different architectures and inventions have made it easier

LeNet (1998)

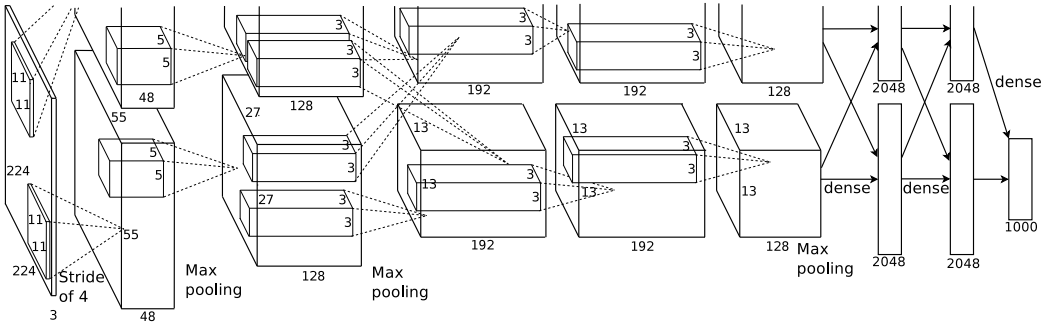
- Handwritten Digit Recognition with a Back-Propagation Network, Le Cun et al.
- First convolutional network
- 2 conv layers, 2 pooling (averaging) layers



AlexNet (2012)

- 5 Convolution Layers,
- Max-Pool Layers
- 3 Fully connected layers at the end
- RELU activation
- 60 million parameters (!)

- Was trained on multiple GTX 580 GPUs with only 3GB Memory



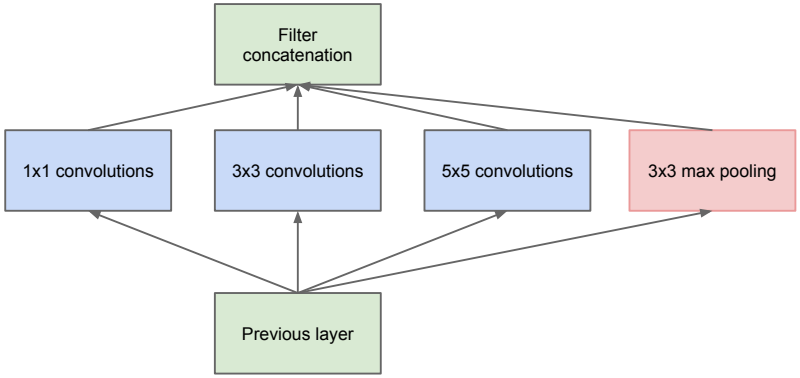
Full (simplified) AlexNet Architecture

[227x227x3] INPUT
[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer
[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer
[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2
[4096] FC6: 4096 neurons
[4096] FC7: 4096 neurons
[1000] FC8: 1000 neurons (class scores)

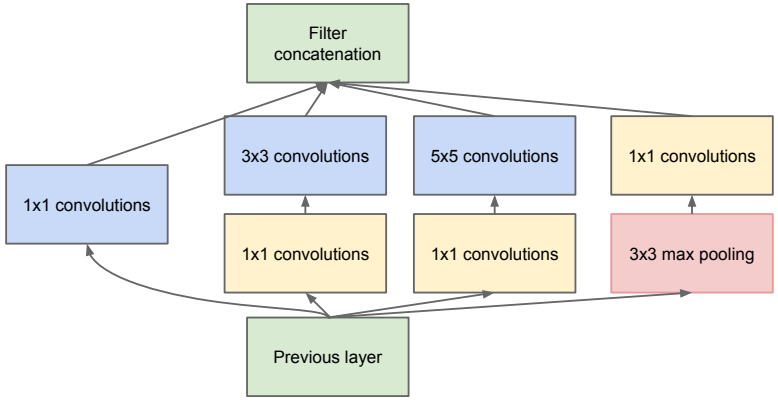
GoogLeNet: Inception Architecture

GoogLeNet: Motivation

- Larger and deeper nets are necessary
- New architecture with filters of various sizes



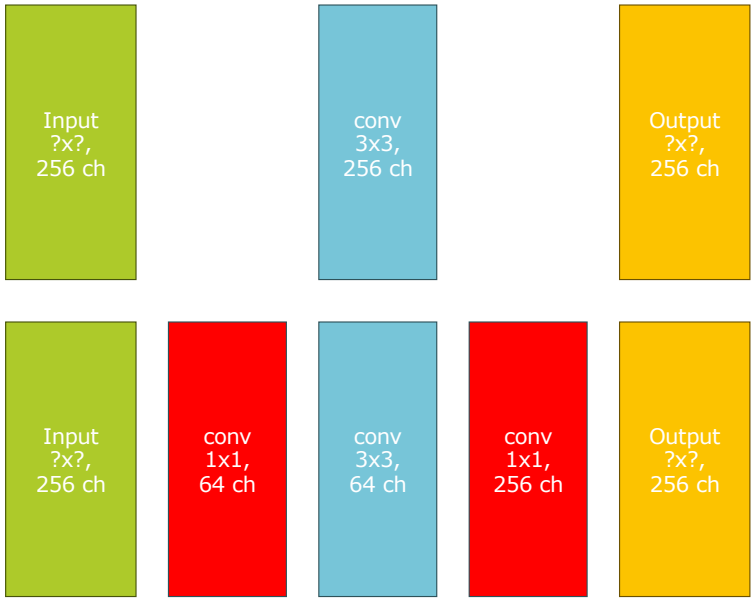
naive version



version with dimensionality reduction (bottleneck layers)

Bottleneck layers

A bottleneck layer is a 1x1 convolutional layer from from a higher number of channels to a lower number (or vice versa)



Parameters:

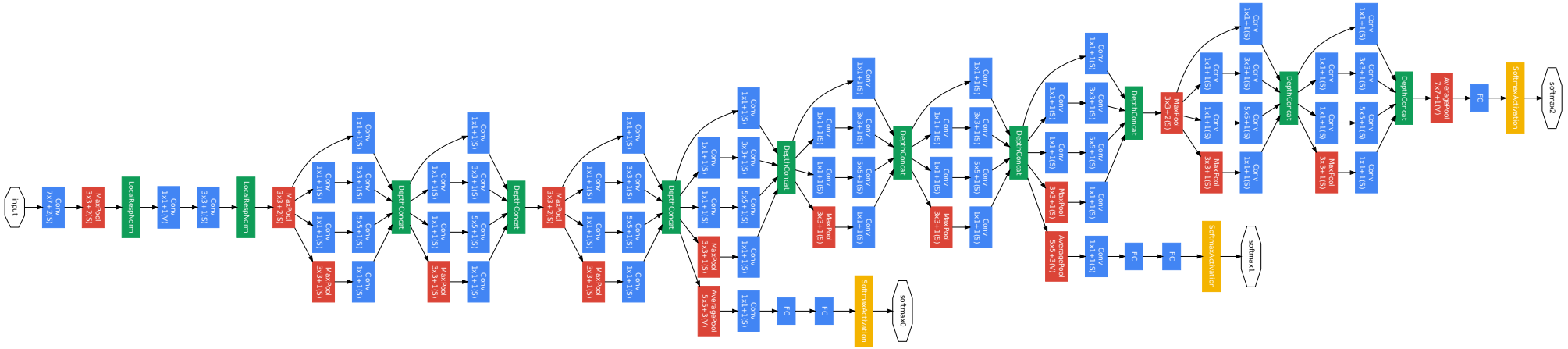
(3*3*256 +1) * 256 = 590080

(1*1*256 + 1) * 64 = 16448
(3*3*64 + 1) * 64 = 36928
(1*1*64 + 1) * 256 = 16640

Total: 70016

GoogLeNet: The Inception Network

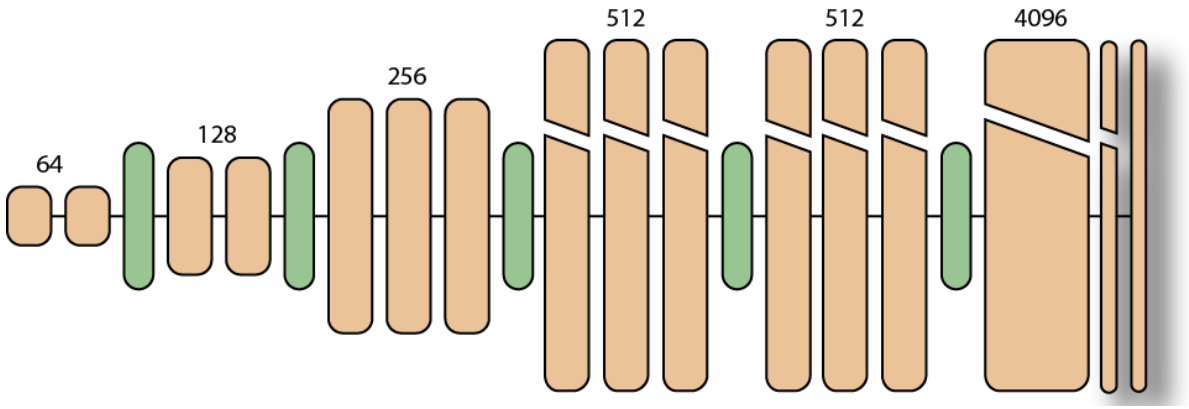
- 22 layers deep (layers with parameters)
- ca. 100 layer building blocks



VGG Net

- Simonyan & Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*

- Examine effect of depth of networks
- 3x3 (!) convolutional layers only, some with pooling

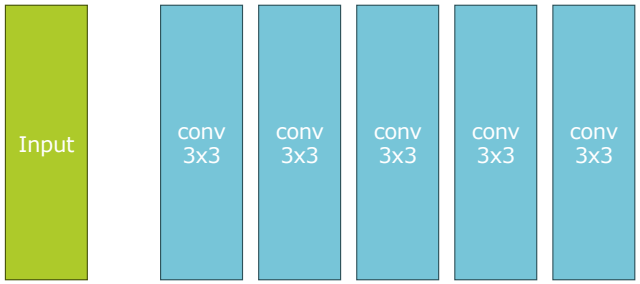


Replace large filter kernels with a sequence of smaller filters

Receptive field:

Parameters:

11 x 11



50

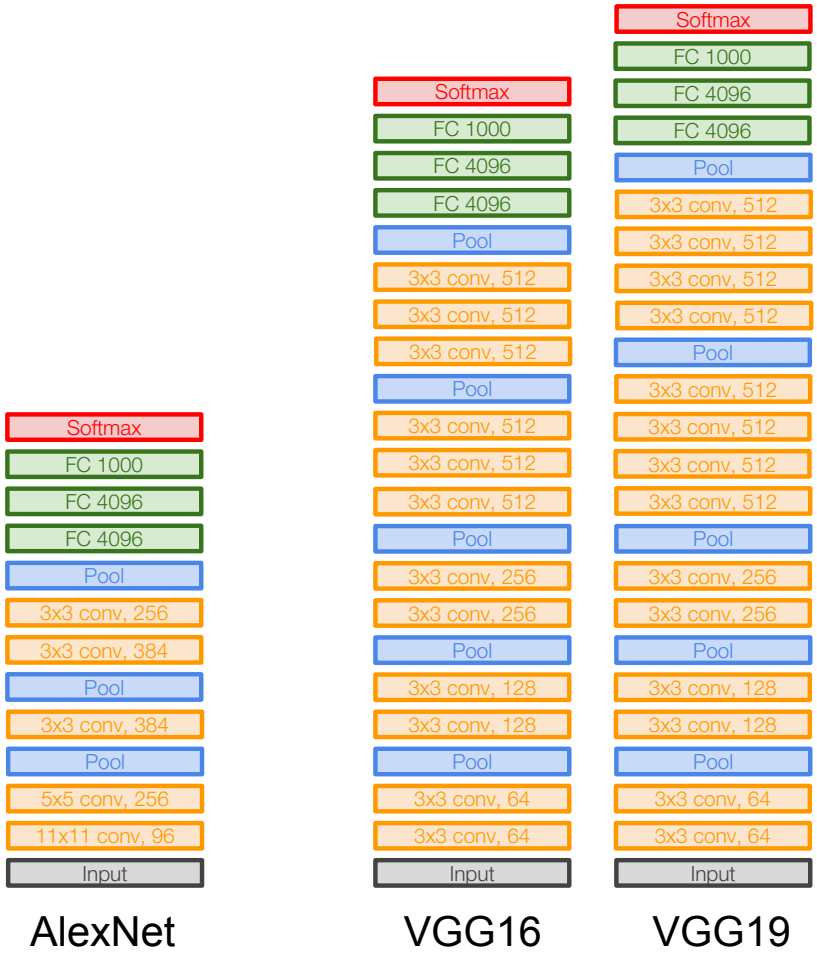
11 x 11

122

VGG Nets

Why use small filters?

- Stack of 5 3x3 filters has the same receptive field as a 11x11 filters
- Less parameters
- More non-linearity
- 27 vs 49 weights (not counting biases) per channel



VGG Net 16

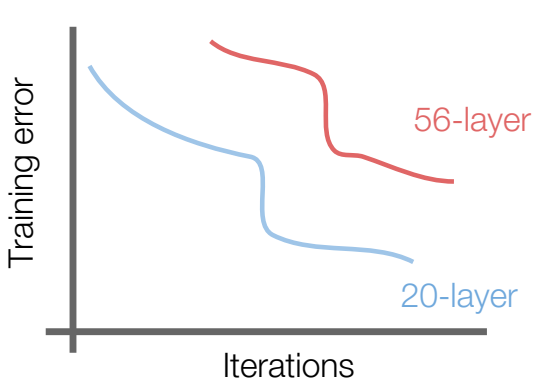
INPUT: [224x224x3] memory: 224*224*3=150K params: 0
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*64)*64 = 36,864
POOL2: [112x112x64] memory: 112*112*64=800K params: 0
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456
POOL2: [56x56x128] memory: 56*56*128=400K params: 0
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
POOL2: [28x28x256] memory: 28*28*256=200K params: 0
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512] memory: 14*14*512=100K params: 0
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512] memory: 7*7*512=25K params: 0
FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = 102,760,448
FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

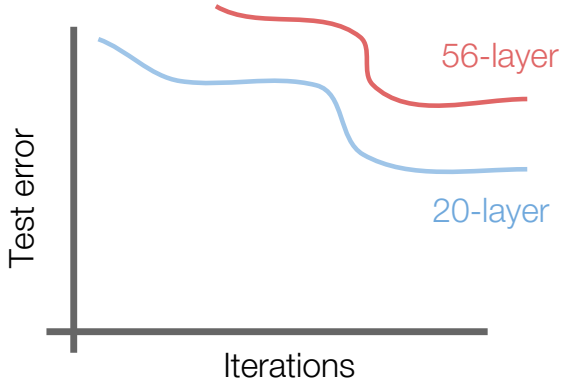
VGG Net 16

INPUT: [224x224x3] memory: 224*224*3=150K params: 0
CONV3-64: [224x224x64] memory: 224*224*64=**3.2M** params: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64] memory: 224*224*64=**3.2M** params: (3*3*64)*64 = 36,864
POOL2: [112x112x64] memory: 112*112*64=800K params: 0
CONV3-128: [112x112x128] memory: 112*112*128=**1.6M** params: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128] memory: 112*112*128=**1.6M** params: (3*3*128)*128 = 147,456
POOL2: [56x56x128] memory: 56*56*128=400K params: 0
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
POOL2: [28x28x256] memory: 28*28*256=200K params: 0
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512] memory: 14*14*512=100K params: 0
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512] memory: 7*7*512=25K params: 0
FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = **102,760,448**
FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

Residual Networks: ResNet

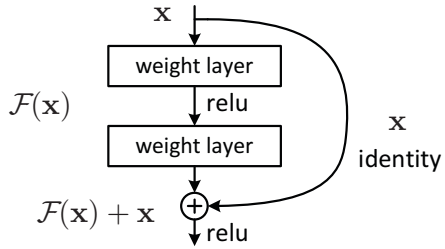
- What happens when we add layers to a convolutional network?
- 56 layer model is worse than 20 layer, but not from overfitting



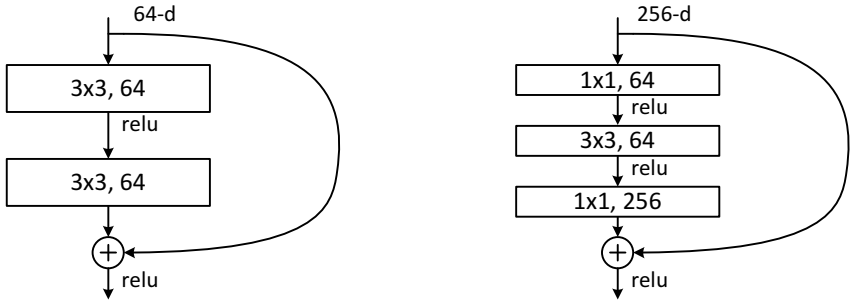


Residual connection

- Residual or skip connections **add** the **input** of a block of layers to its **output**
- This helps to propagate the gradient backwards, as layers are initialized close to zero
- For deeper networks, the connections can be combined with bottleneck layers

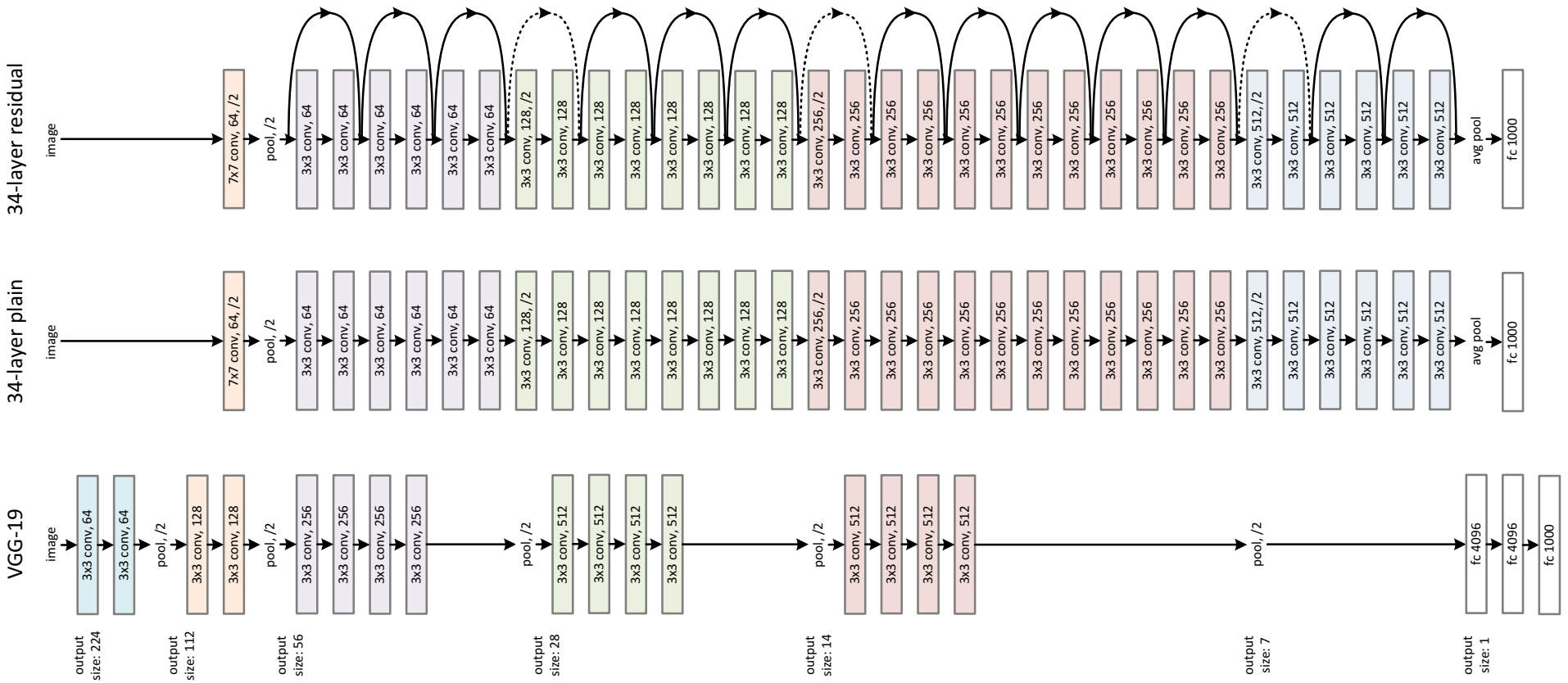


Original building block

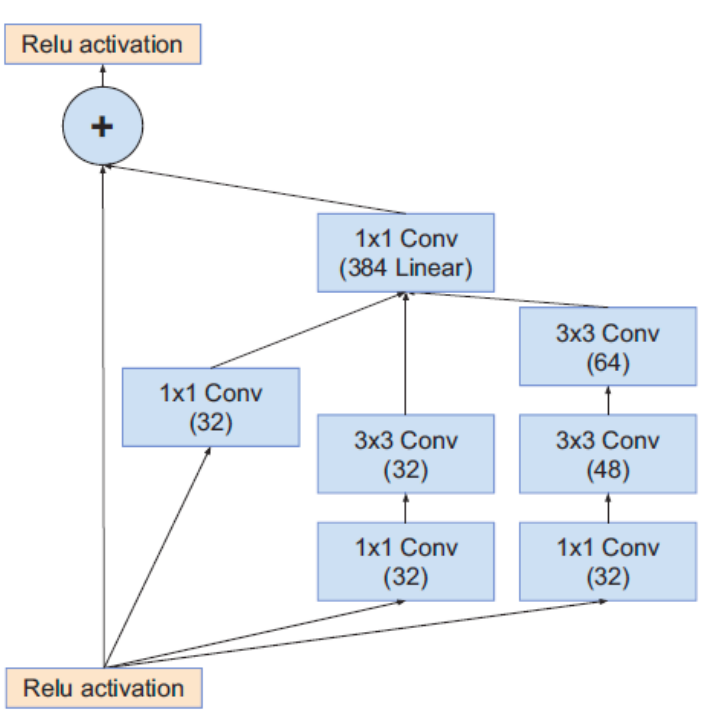


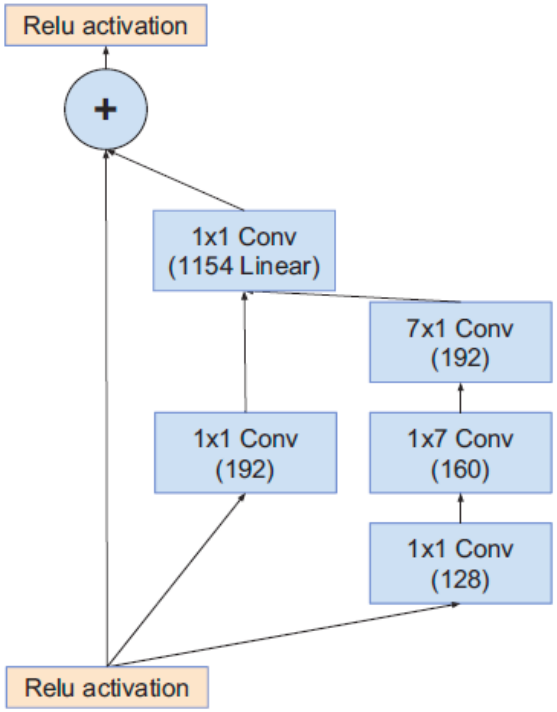
Building blocks for resnet-34 (left), and resnet-101 (right) with bottleneck layers

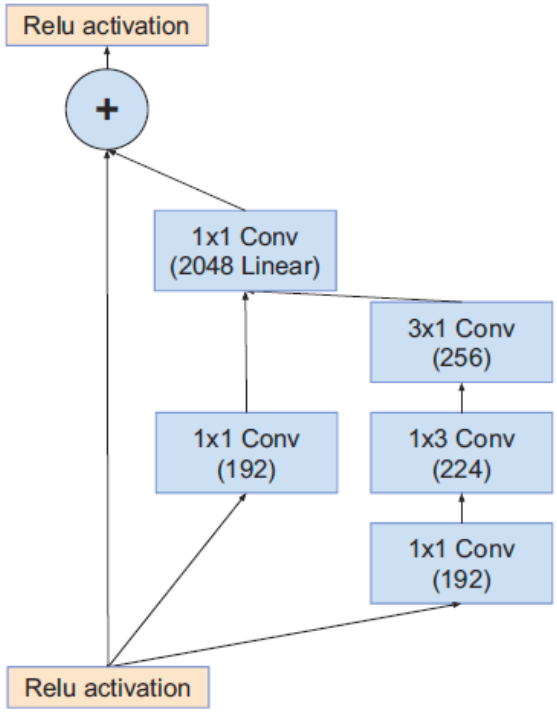
The ResNet 34



Inception ResNet v2







Layer Normalization

Problem:

- Training deep neural networks is difficult and sometimes does not converge
- While the data inputs are normalized, the inputs to later layers in the network are not (depending on the used activation function)

Solution

- Batch Normalization to learn and adapt normalization of inputs

Batch Normalization

$$O_{b,c,x,y} \leftarrow \gamma_c \frac{I_{b,c,x,y} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c \quad \forall b, c, x, y$$

- Normalize the input to layers
- Accelerates training
- Enables higher learning rates
- Improves Generalization
- Parameters are learned from mini-batches

BatchNorm2d

CLASS torch.nn.BatchNorm2d(*num_features*, *eps=1e-05*, *momentum=0.1*, *affine=True*, *track_running_stats=True*, *device=None*, *dtype=None*) [SOURCE]

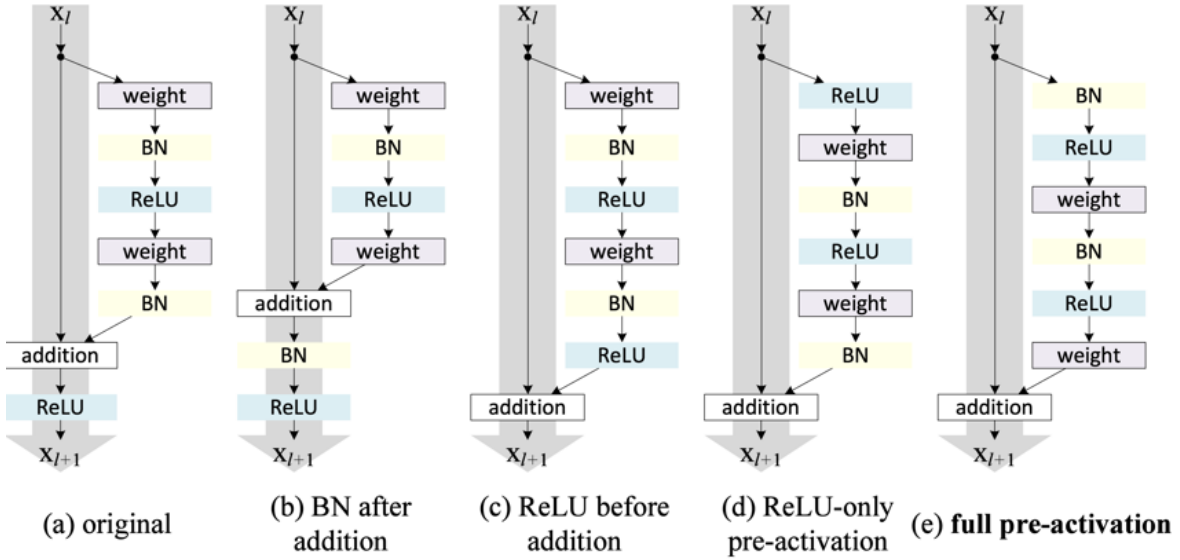
Applies Batch Normalization over a 4D input.

4D is a mini-batch of 2D inputs with additional channel dimension. Method described in the paper Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift .

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

Residual connections and batch normalization

case	Fig.	ResNet-110	ResNet-164
original Residual Unit [1]	Fig. 4(a)	6.61	5.93
BN after addition	Fig. 4(b)	8.17	6.50
ReLU before addition	Fig. 4(c)	7.84	6.14
ReLU-only pre-activation	Fig. 4(d)	6.71	5.91
full pre-activation	Fig. 4(e)	**6.37**	**5.46**



ImageNet Winners

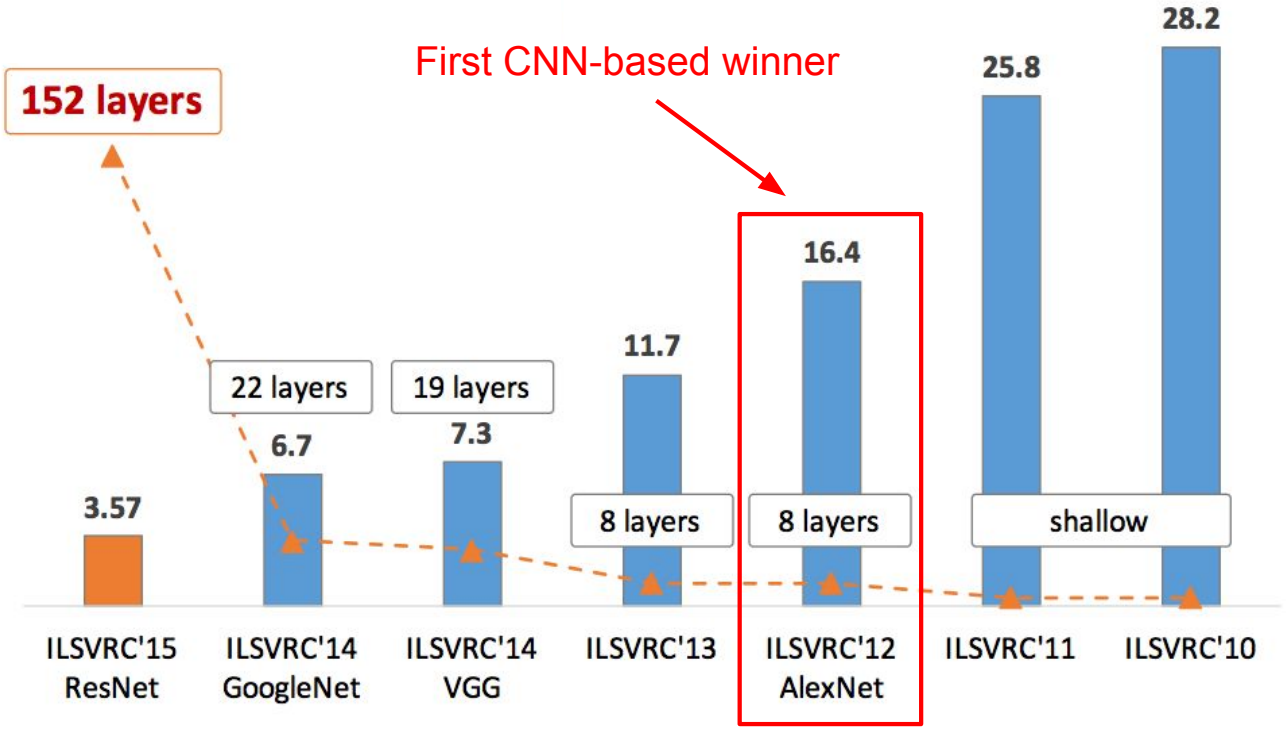
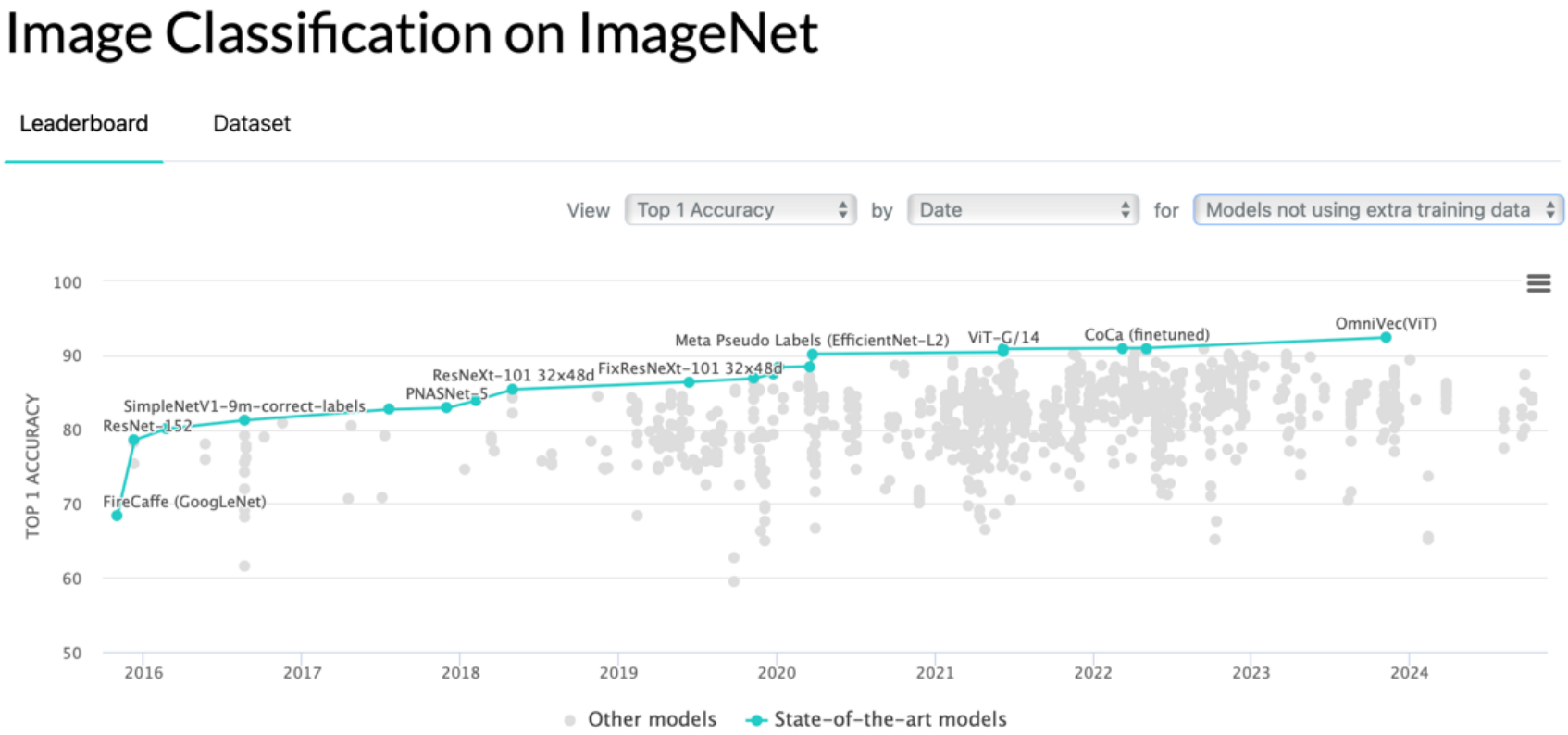


Figure copyright Kaiming He, 2016.

ImageNet: Newest results

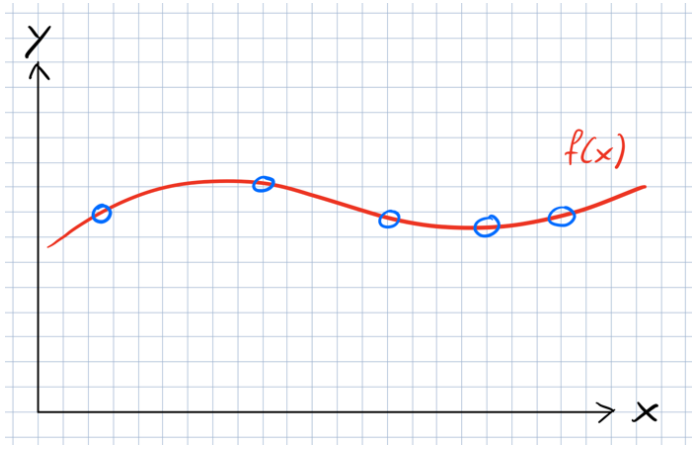


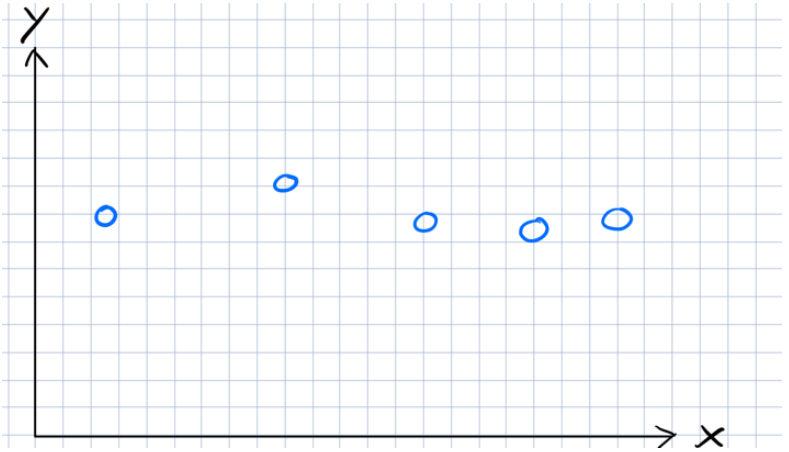
Training Deep Networks

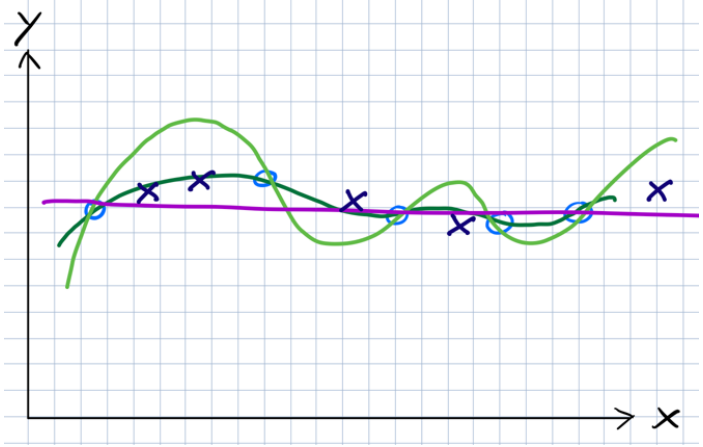
How deep and large should your network be?

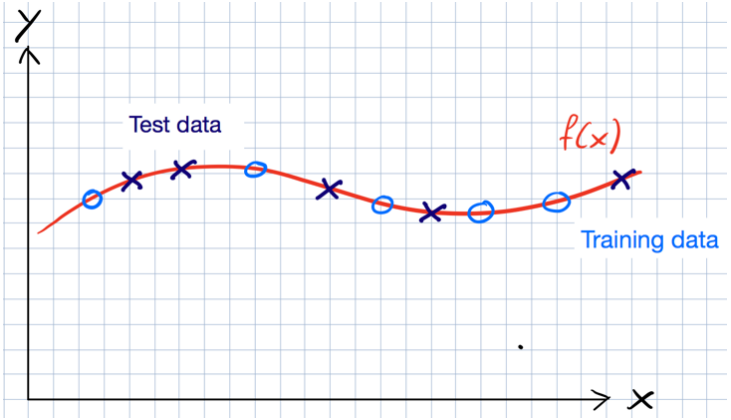
How can we train efficiently?

Example









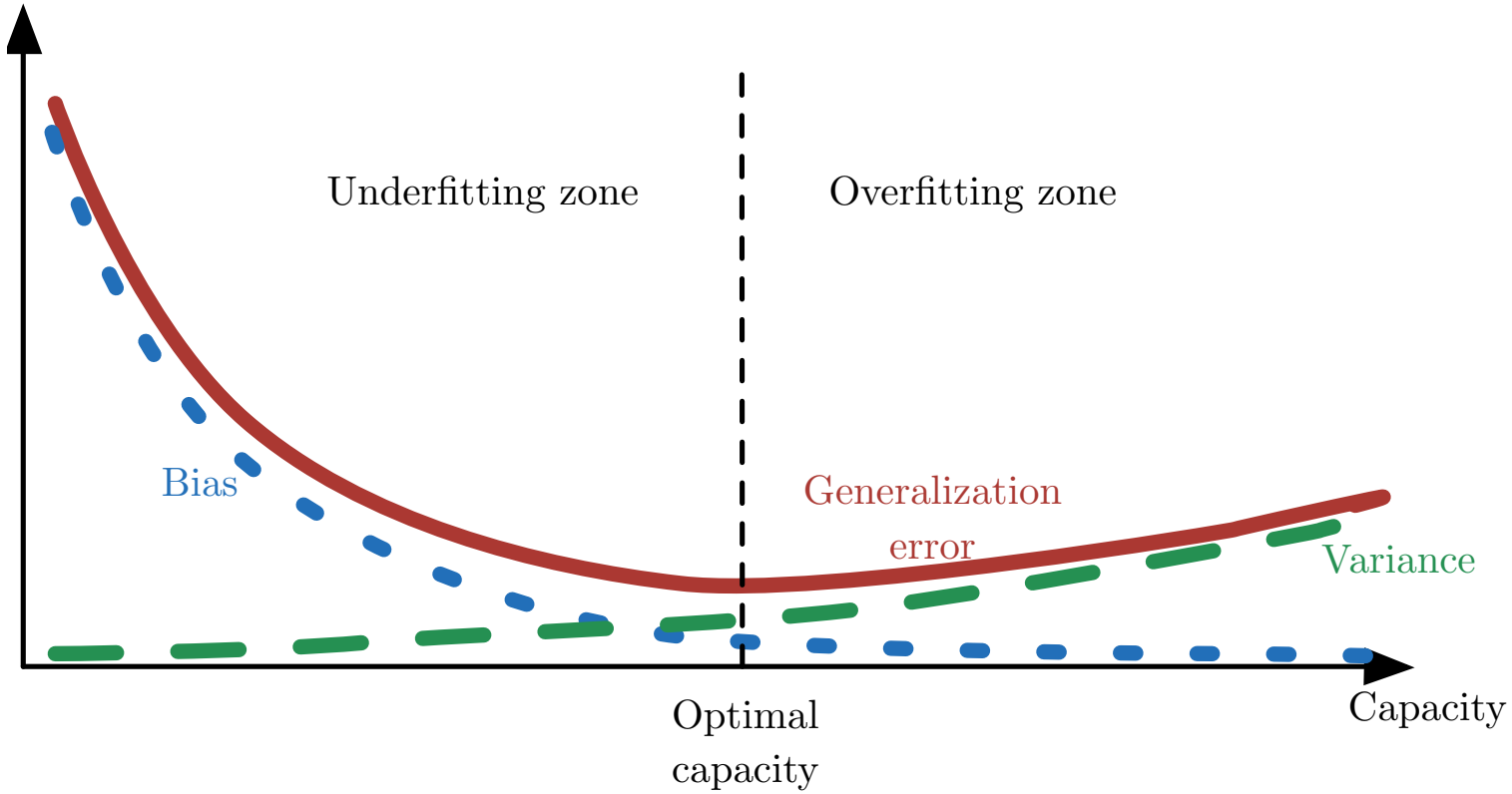
Errors

Training Error:	Error on training set
Validation Error:	Error on validation set
Generalization Error:	Gap between the error on the training and validation sets

Data Sets for training:

Training data set:	Data Set to run your training on
Validation data set:	Data Set to evaluate your trained model on and tune hyper parameters
Test data set:	Evaluation of final model

Optimal Capacity



Constrain model parameters

Problem:

- Add constraints to the model to favor less complex models, while keeping the depth

Effect:

- Complex models suffer from high variance error which leads to poor generalisation

Solution

- Regularisation, for example L2 or Dropout

L2 Regularization

Full loss function includes regularization over all parameters:

Classic view:

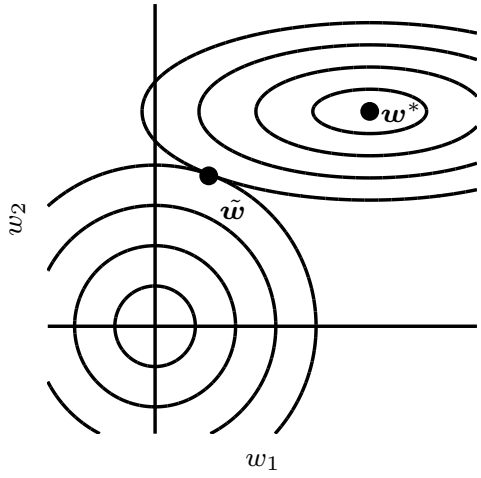
- Regularization works to prevent overfitting when we have a lot of features (or later a very powerful/deep model, etc.)

Now:

- Regularization produces models that generalize well when we have a “big” model
- We do not care that our models overfit on the training data, even though they are hugely overfit

L2 Regularization

Limit the capacity of networks by adding a parameter norm penalty to the loss



Goodfellow 2016

$$\tilde{J}(\mathbf{w}; X, y) = J(\mathbf{w}; X, y) + \alpha\Omega(\mathbf{w})$$

$$\Omega(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|_2^2$$

$$\tilde{J}(\mathbf{w}; X, y) = \frac{\alpha}{2}\mathbf{w}^T\mathbf{w} + J(\mathbf{w}; X, y)$$

$$\nabla_w \tilde{J}(\mathbf{w}; X, y) = \alpha\mathbf{w} + \nabla_w J(\mathbf{w}; X, y)$$

Dropout

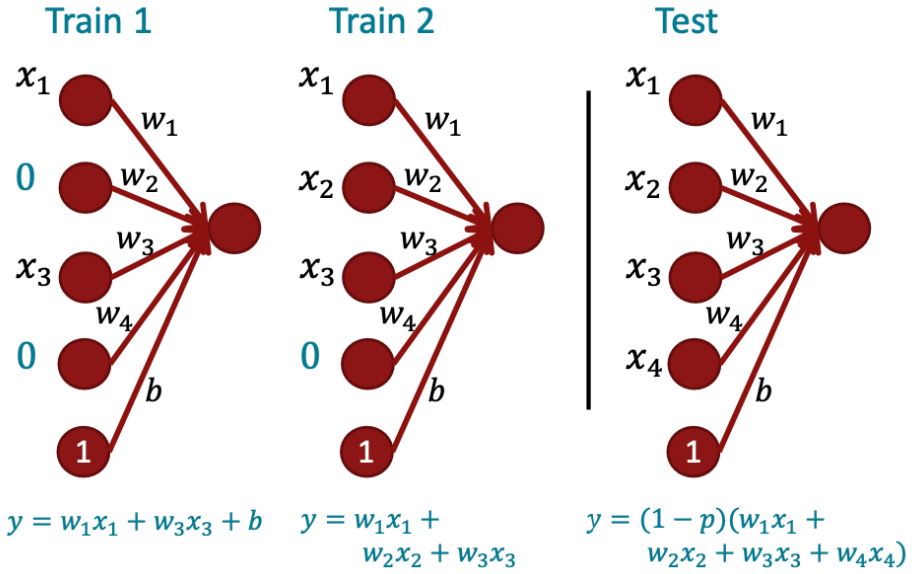
During training:

- Randomly set input to 0 with probability p (dropout ratio)

During testing / evaluation

- Multiply all weights by 1-p

- (Alternatively multiply weights at training with 1 / (1-p))

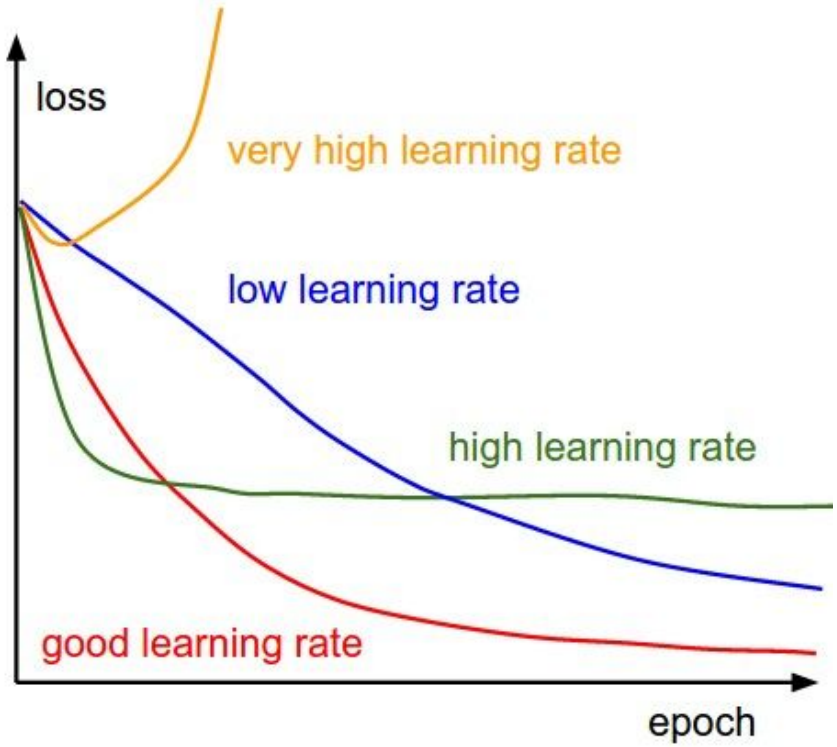


Hyperparameter Optimization

First **coarse** search of parameters, then finer search

Most important hyperparameters to try out:

- Network capacity/architecture
- Learning rate, decay rate if used
- Regularization weights
- Dropout percentage (if used)



Convolutional Neural Networks in Practice: Fine-tuning existing models

The discussed models were trained on the **ImageNet dataset**, which contains more than **14 million labeled images**!

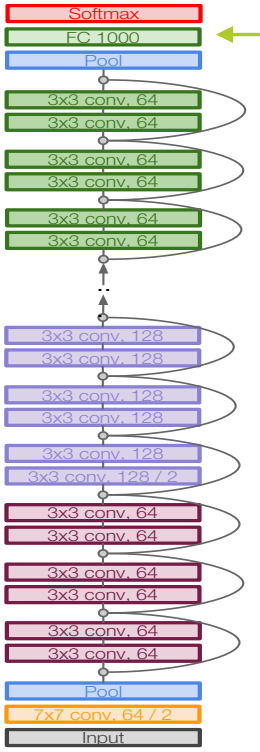
In practice, you rarely have access to labeled datasets of this size.

What to do?

- Fine-tuning an existing model

Available models

Model	Size	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth
Xception	88 MB	0.790	0.945	22,910,480	126
VGG16	528 MB	0.713	0.901	138,357,544	23
VGG19	549 MB	0.713	0.900	143,667,240	26
ResNet50	98 MB	0.749	0.921	25,636,712	-
ResNet101	171 MB	0.764	0.928	44,707,176	-
ResNet152	232 MB	0.766	0.931	60,419,944	-
ResNet50V2	98 MB	0.760	0.930	25,613,800	-
ResNet101V2	171 MB	0.772	0.938	44,675,560	-
ResNet152V2	232 MB	0.780	0.942	60,380,648	-
InceptionV3	92 MB	0.779	0.937	23,851,784	159
InceptionResNetV2	215 MB	0.803	0.953	55,873,736	572
MobileNet	16 MB	0.704	0.895	4,253,864	88
MobileNetV2	14 MB	0.713	0.901	3,538,984	88
DenseNet121	33 MB	0.750	0.923	8,062,504	121
DenseNet169	57 MB	0.762	0.932	14,307,880	169
DenseNet201	80 MB	0.773	0.936	20,242,984	201
NASNetMobile	23 MB	0.744	0.919	5,326,716	-
NASNetLarge	343 MB	0.825	0.960	88,949,818	-
EfficientNetB0	29 MB	-	-	5,330,571	-
EfficientNetB1	31 MB	-	-	7,856,239	-
EfficientNetB2	36 MB	-	-	9,177,569	-
EfficientNetB3	48 MB	-	-	12,320,535	-
EfficientNetB4	75 MB	-	-	19,466,823	-
EfficientNetB5	118 MB	-	-	30,562,527	-
EfficientNetB6	166 MB	-	-	43,265,143	-
EfficientNetB7	256 MB	-	-	66,658,687	-



Retrain only this layer with your data

- Learning few parameters requires few training samples
- Assumption: the ”stem” of the network learned to extract generally useful features

Reuse as-is

Convolutional Neural Networks in Practice: Augmenting your data set

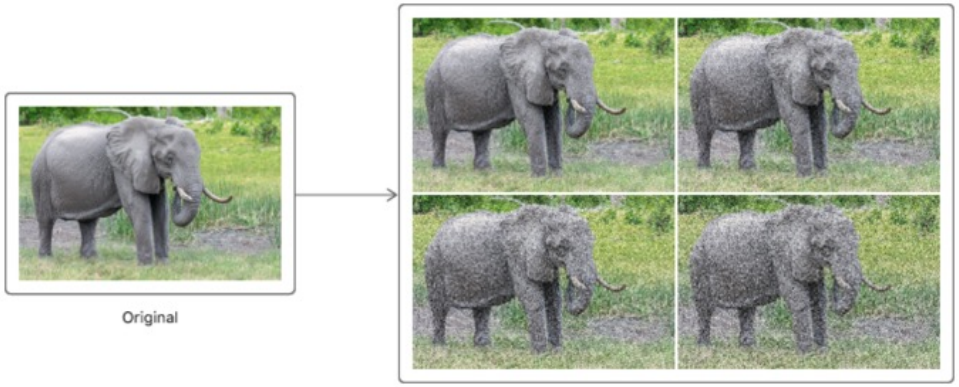
The discussed models were trained on the **ImageNet dataset**, which contains more than **14 million labeled images**!

In practice, you rarely have access to labeled datasets of this size.

What to do?

- Fine-tuning an existing model
- Augmenting your Dataset

Source: https://medium.com/analytics-vidhya/data-augmentation-is-it-really-necessary-b3cb12ab3c3f



Source: https://developer.apple.com/documentation/createml/mlimageclassifier/imageaugmentationoptions/noise

Convolutional Neural Networks in Practice: Augmenting your Dataset

The discussed models were trained on the **ImageNet dataset**, which contains more than **14 million labeled images**!

In practice, you rarely have access to labeled datasets of this size.

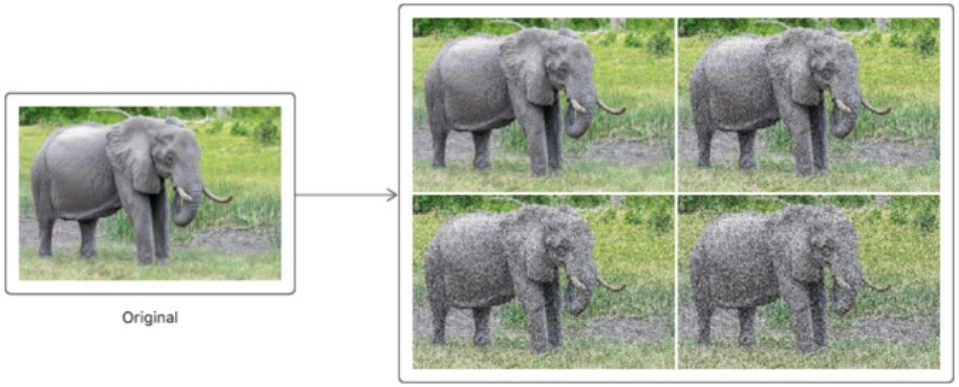
What to do?

- Fine-tuning an existing model and/or
- Augmenting your Dataset

As long as a human can classify the image, the CNN should be able to do too!

It also makes the CNN more robust!

Source: https://medium.com/analytics-vidhya/data-augmentation-is-it-really-necessary-b3cb12ab3c3f



Source: https://developer.apple.com/documentation/createml/mlimageclassifier/imageaugmentationoptions/noise

Exercise: Classification on the CIFAR-10 Dataset

This time with CNNs ☺